

CENTRO UNIVERSITÁRIO – CATÓLICA DE SANTA CATARINA
CURSO DE ENGENHARIA DE SOFTWARE
GUSTAVO HENRIQUE ANTUNES DE LIMA
LUCAS FERNANDES
RAQUEL DA SILVA

RELATÓRIO DA N3 DE FERRAMENTAS WEB E UX: PROJETO UTILIZANDO AS
HEURÍSTICAS DE NIELSEN

JARAGUÁ DO SUL
JULHO 2026

**GUSTAVO HENRIQUE ANTUNES DE LIMA
LUCAS FERNANDES
RAQUEL DA SILVA**

**RELATÓRIO DA N3 DE FERRAMENTAS WEB E UX: PROJETO UTILIZANDO AS
HEURÍSTICAS DE NIELSEN**

Relatório apresentado à disciplina de Ferramentas Web e Ux do Curso de Engenharia de Software, do Centro Universitário – Católica de Santa Catarina.

Professor Orientador: **Luiz Almeida Junior**

**JARAGUÁ DO SUL
JULHO 2026**

SUMÁRIO

1	INTRODUÇÃO.....	3
2	HEURÍSTICAS MAIS IMPORTANTES PARA O PROJETO.....	4
3	PROBLEMAS DE USABILIDADE ENCONTRADOS.....	5
	3.1 AUSÊNCIA DE FEEDBACK DURANTE CARREGAMENTO.....	5
	3.2 EXCLUSÃO SEM CONFIRMAÇÃO.....	5
	3.3 VALIDAÇÃO DE EMAIL INEXISTENTE.....	5
	3.4 CAMPOS SEM PLACE HOLDER.....	5
	3.5 CARDS COM TÍTULO SEM CAPITALIZAÇÃO.....	5
	3.6 BOTÃO DE NAVEGAÇÃO SEM INDICAÇÃO DE PÁGINA ATIVA.....	6
4	MELHORIAS REALIZADAS.....	7
	4.1 INDICADOR DE CARREGAMENTO(SPINNER).....	7
	4.2 MODAL DE CONFIRMAÇÃO PARA EXCLUSÃO.....	7
	4.3 VALIDAÇÃO DE FORMATO DE E-MAIL.....	7
	4.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO.....	7
	4.5 CAPITALIZAÇÃO DOS TÍTULOS DOS CARDS.....	7
	4.6 DESTAQUE DO BOTÃO DE NAVEGAÇÃO.....	7
5	COMO AS MELHORIAS FORAM IMPLEMENTADAS.....	8
	5.1 INDICADOR DE CARREGAMENTO.....	8
	5.2 MODAL DE CONFIRMAÇÃO DE EXCLUSÃO.....	9
	5.3 VALIDAÇÃO DO EMAIL.....	11
	5.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO.....	12
	5.5 CAPITALIZAÇÃO DOS TEXTOS DOS CARDS.....	12
	5.6 DESTAQUE AO BOTÃO DE NAVEGAÇÃO ATIVO.....	13
6	QUAIS BENEFÍCIOS TROUXERAM PARA O USUÁRIO.....	15
	6.1 SPINNER DE CARREGAMENTO.....	15
	6.2 MODAL DE CONFIRMAÇÃO PARA EXCLUSÃO.....	15
	6.3 VALIDAÇÃO DE FORMATO DE E-MAIL.....	15
	6.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO.....	15
	6.5 CAPITALIZAÇÃO DOS TÍTULOS DOS CARDS.....	15
	6.6 DESTAQUES DO BOTÃO DE NAVEGAÇÃO ATIVO.....	15

1 INTRODUÇÃO

A usabilidade é um dos principais fatores para garantir que um sistema seja eficiente, intuitivo e agradável para seus usuários. Para avaliar e aperfeiçoar interfaces, as Heurísticas de Nielsen constituem um conjunto de princípios amplamente utilizados na identificação de problemas de interação e na proposição de melhorias voltadas à experiência do usuário.

Este relatório apresenta a análise de usabilidade realizada em uma aplicação web, tomando como base as heurísticas de Nielsen. A partir dessa análise, foram identificados problemas relacionados ao feedback do sistema, prevenção de erros, navegação e preenchimento de formulários, comprometendo a interação do usuário com a aplicação.

Com base nos problemas encontrados, foram propostas e implementadas melhorias com o objetivo de tornar a interface mais intuitiva, segura e eficiente. Entre as modificações realizadas destacam-se a implementação de um indicador de carregamento (spinner), a criação de um modal de confirmação para exclusão de registros, a validação do formato de e-mail, a inserção de placeholders nos campos dos formulários, a padronização da capitalização dos textos dos cards e o destaque visual da seção ativa da aplicação.

Por fim, o relatório descreve as heurísticas consideradas mais relevantes para o projeto, apresenta os problemas de usabilidade identificados, detalha as soluções implementadas e discute os benefícios proporcionados por cada uma das melhorias para a experiência do usuário.

2 HEURÍSTICAS MAIS IMPORTANTES PARA O PROJETO

As heurísticas de mais importância para o projeto são aquelas em que precisamos melhorar, sendo elas:

Heurística nº 1 — Visibilidade do Status do Sistema: O sistema deve manter o usuário continuamente informado sobre o que está ocorrendo, por meio de feedbacks adequados e fornecidos em tempo oportuno, como barras de progresso e indicadores de carregamento.

Heurística nº 3 — Controle e Liberdade do Usuário: O usuário deve ter a possibilidade de desfazer e refazer ações com facilidade, contando com uma "saída de emergência" clara para situações em que cometa erros, como o atalho "Ctrl+Z" ou botões de cancelamento.

Heurística nº 5 — Prevenção de Erros: Mais eficaz do que apresentar mensagens de erro é desenvolver um sistema que evite sua ocorrência, eliminando condições propícias a falhas ou solicitando confirmação do usuário antes da execução de ações irreversíveis.

Heurística nº 6 — Reconhecimento em Vez de Memorização: A carga cognitiva do usuário deve ser minimizada por meio da exposição clara de objetos, ações e opções, dispensando a necessidade de memorizar informações entre etapas da interação.

Heurística nº 8 — Estética e Design Minimalista: As interfaces não devem conter informações irrelevantes ou dispensáveis, uma vez que cada elemento desnecessário compete com o conteúdo relevante, reduzindo sua visibilidade e clareza.

Heurística nº 9 — Suporte ao Reconhecimento, Diagnóstico e Correção de Erros: As mensagens de erro devem ser redigidas em linguagem simples e acessível, sem códigos técnicos, indicando com precisão a natureza do problema e orientando o usuário na sua resolução.

3 PROBLEMAS DE USABILIDADE ENCONTRADOS

Os problemas que foram identificados no trabalho serão apresentados de uma maneira que será falado sobre qual o problema de forma resumida e quais heurísticas estão indo contra.

3.1 AUSÊNCIA DE FEEDBACK DURANTE CARREGAMENTO

Ao clicar em 'Posts' ou 'Users', a aplicação fazia uma requisição à API sem nenhum indicador visual. O usuário ficava sem saber se a ação havia funcionado, se estava carregando ou se havia ocorrido um erro — violando diretamente a heurística nº1 de visibilidade do status do sistema.

3.2 EXCLUSÃO SEM CONFIRMAÇÃO

O botão 'Delete' removerá imediatamente um card da lista sem qualquer diálogo de confirmação. Um clique acidental resultava em perda irreversível de dados, ferindo a heurística de nº3 controle e liberdade do usuário.

3.3 VALIDAÇÃO DE EMAIL INEXISTENTE

O campo de e-mail no formulário de usuários aceitava qualquer texto, sem verificar se o formato era válido (ex: 'abc' era aceito como e-mail). Além disso, ao errar, o alerta nativo do navegador (alert()) oferecia mensagem genérica, sem indicar qual campo estava incorreto. Ferindo assim as heurísticas nº5 prevenção de erros e a nº9 suporte ao reconhecimento, diagnóstico e correção de erros.

3.4 CAMPOS SEM PLACE HOLDER

Os inputs do formulário possuíam apenas labels externas. Sem texto de exemplo dentro do campo, o usuário não tinha referência do formato esperado para cada dado. Inferindo a heurísticas de nº6 reconhecimento ao invés de memorização.

3.5 CARDS COM TÍTULO SEM CAPITALIZAÇÃO

Os títulos dos posts vinham em letras minúsculas da API, sem nenhum tratamento de capitalização, resultando em aparência descuidada e inconsistente nos cards. Que infere diretamente na heurística nº8 estética e design minimalista.

3.6 BOTÃO DE NAVEGAÇÃO SEM INDICAÇÃO DE PÁGINA ATIVA

Os botões 'Posts' e 'Users' no menu de navegação não tinham nenhum destaque visual para indicar qual seção estava ativa no momento, dificultando a orientação do usuário dentro da aplicação. Inferindo assim na heurística nº1 visibilidade do status do sistema e nº6 reconhecimento em vez de memorização.

4 MELHORIAS QUE FORAM REALIZADAS

4.1 INDICADOR DE CARREGAMENTO (SPINNER)

Foi adicionado um spinner que aparece durante as requisições à API (ao clicar em "Posts" ou "Users"), sendo removido automaticamente quando os dados chegam. Resolve a ausência de feedback visual durante o carregamento.

4.2 MODAL DE CONFIRMAÇÃO PARA EXCLUSÃO

A exclusão direta de itens foi substituída por um modal que pergunta "Tem certeza que deseja excluir este item?", exigindo confirmação do usuário antes de remover o dado permanentemente.

4.3 VALIDAÇÃO DE FORMATO DE E-MAIL

Foi implementada uma expressão regular para validar o campo de e-mail no formulário de usuários. Quando o formato é inválido, uma mensagem específica é exibida, apontando o erro.

4.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO

Todos os inputs passaram a ter texto de exemplo dentro do campo (como "ex: joao@email.com"), servindo de referência visual para o preenchimento correto.

4.5 CAPITALIZAÇÃO DOS TÍTULOS DOS CARDS

Uma função de capitalização foi aplicada aos títulos dos posts, deixando a primeira letra maiúscula e tornando a exibição mais consistente.

4.6. DESTAQUE DO BOTÃO DE NAVEGAÇÃO

O botão da seção atualmente selecionada ("Posts" ou "Users") recebe uma classe CSS de destaque (cor de fundo e borda), indicando claramente ao usuário em que parte da aplicação ele está.

5.1 INDICADOR DE CARREGAMENTO

A primeira melhoria consistiu na implementação de um indicador de carregamento (spinner), utilizado para informar ao usuário que os dados estão sendo processados. Para isso, foi criada a classe spinner no arquivo CSS, responsável por definir o formato circular do componente e sua animação de rotação contínua. A animação foi implementada por meio da regra `@keyframes`, que estabelece a rotação do elemento de 0° a 360°. No JavaScript, foi desenvolvida a função `show_spinner()`, responsável por inserir dinamicamente o componente na área de exibição dos cartões antes da execução da requisição (fetch). Dessa forma, o usuário recebe um retorno visual durante o carregamento das informações.

Figura 1 - Tela principal com o indicador de carregamento



Figura 2 - Código CSS da classe spinner

```
.spinner {  
  width: 48px;  
  height: 48px;  
  border: 5px solid #f3f3f3;  
  border-top: 5px solid #7d0f0f;  
  border-radius: 50%;  
  animation: spin 0.8s linear infinite;  
  margin: 60px auto;  
}  
  
@keyframes spin {  
  0% { transform: rotate(0deg); }  
  100% { transform: rotate(360deg); }  
}
```

Figura 3 - Código javascript da função show_spinner

```
function show_spinner() {  
  const cardContent = document.querySelector('.card-content');  
  cardContent.innerHTML = '<div class="spinner"></div>';  
}
```

Figura 4 - Código javascript de exemplo de uso da função show_spinner

```
function get_user(){
  show_spinner();
  fetch("https://jsonplaceholder.typicode.com/users")
    .then(function(dado_retornado_user){
      return dado_retornado_user.json()
    }).then(function(json){
      dados_user.push(...json);
      create_user_cards();
    })
}
```

5.2 MODAL DE CONFIRMAÇÃO DE EXCLUSÃO

Outra melhoria implementada foi a criação de um modal de confirmação para exclusão de registros, evitando remoções acidentais. Para isso, foi adicionada ao HTML uma estrutura contendo a mensagem de confirmação e os botões "Confirmar" e "Cancelar". No JavaScript, foi utilizada uma variável global para armazenar temporariamente o identificador do registro que será excluído. Ao selecionar a opção de exclusão, essa variável recebe o respectivo ID e o modal é exibido por meio da alteração da propriedade display para flex. Caso o usuário confirme a operação, é executada a lógica de exclusão já existente, seguida do fechamento do modal e da limpeza da variável utilizada. Caso a operação seja cancelada, o modal é ocultado, restaurando a variável para um estado nulo. No CSS, o componente foi configurado inicialmente com display: none, além das propriedades position: fixed, inset: 0 e z-index, garantindo que o modal permaneça centralizado e sobreposto aos demais elementos da interface.

Figura 5 - Modal de confirmação de exclusão

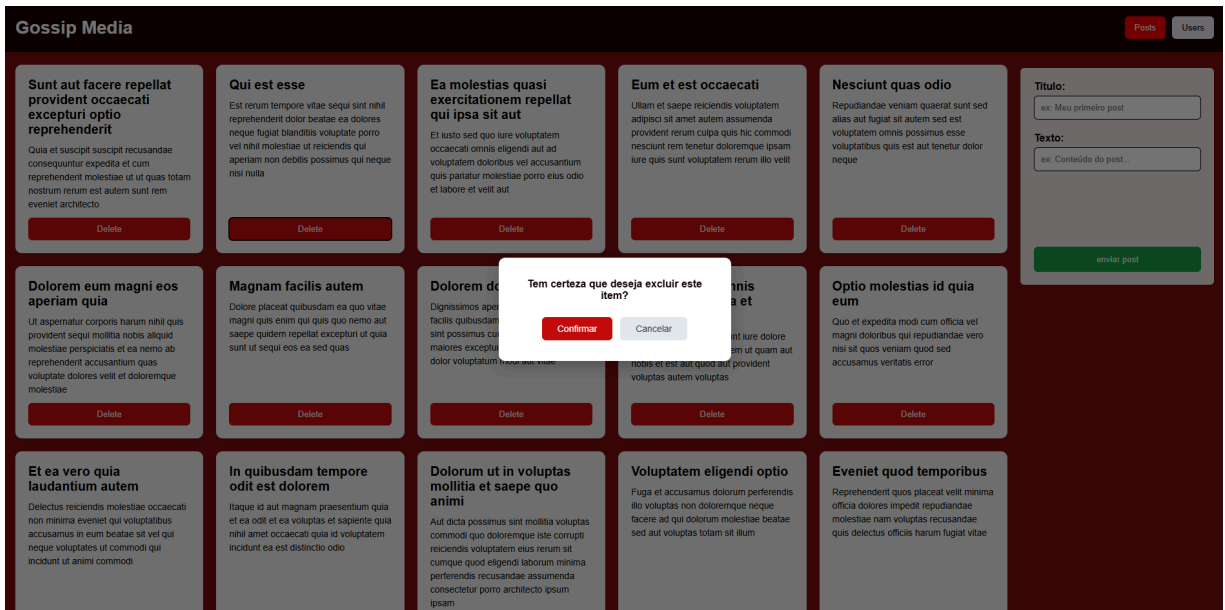


Figura 6 - Estrutura HTML do modal

```
<div id="modal-overlay">
  <div id="modal-box">
    <p>Tem certeza que deseja excluir este item?</p>
    <div id="modal-buttons">
      <button id="modal-confirm">Confirmar</button>
      <button id="modal-cancel">Cancelar</button>
    </div>
  </div>
</div>
```

Figura 7 - Código javascript da ação ao clicar no botão de exclusão

```
document.getElementById('modal-confirm').addEventListener('click', function () {
  if (pendingDeleteId === null) return;
  let index;
  if (currentType === 'posts') {
    index = dados_post.findIndex(item => item.id === pendingDeleteId);
    dados_post.splice(index, 1);
    create_post_cards();
  } else if (currentType === 'users') {
    index = dados_user.findIndex(item => item.id === pendingDeleteId);
    dados_user.splice(index, 1);
    create_user_cards();
  }
  pendingDeleteId = null;
  document.getElementById('modal-overlay').style.display = 'none';
});
```

Figura 8 - Código javascript da ação ao clicar no botão de cancelar

```
document.getElementById('modal-cancel').addEventListener('click', function () {
  pendingDeleteId = null;
  document.getElementById('modal-overlay').style.display = 'none';
});
```

Figura 9 - Código CSS do modal

```
#modal-overlay {
  display: none;
  position: fixed;
  inset: 0;
  background-color: rgba(0, 0, 0, 0.55);
  z-index: 1000;
  justify-content: center;
  align-items: center;
}
```

5.3 VALIDAÇÃO DO EMAIL

Também foi implementada uma validação para o campo de e-mail utilizando uma expressão regular (Regular Expression – Regex). Essa validação verifica se o endereço informado segue o padrão esperado para um e-mail válido. Para isso, foi utilizada a função `test()`, que compara o texto digitado com o padrão definido pela expressão regular. Caso o formato seja considerado inválido, o sistema exibe uma mensagem de alerta ao usuário, impedindo o prosseguimento da operação.

Figura 10 - Alerta de quando email está incorreto

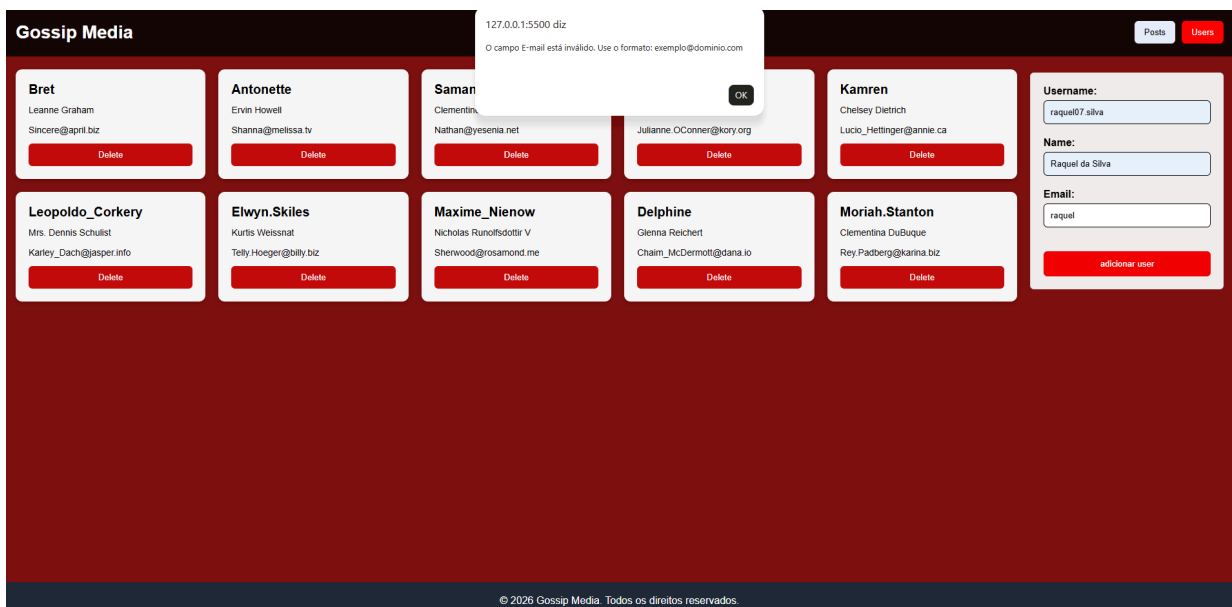


Figura 11 - Código javascript do uso do regex

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(dado_email.value)) {
  alert('O campo E-mail está inválido. Use o formato: exemplo@dominio.com');
  return;
}
```

5.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO

Com o objetivo de melhorar a usabilidade dos formulários, foram adicionados atributos placeholder aos campos de entrada dos formulários de usuários e de publicações. Essa alteração fornece exemplos do conteúdo esperado em cada campo, tornando o preenchimento mais intuitivo e reduzindo possíveis dúvidas durante a utilização do sistema.

Figura 12 - Exemplo no placeholder na tela



Figura 13 - Estrutura HTML do placeholder no form do user

```
function create_user_forms(){
  const formsContent = document.querySelector('.forms-content');
  formsContent.innerHTML = '';
  formsContent.innerHTML+=`
    <form>
      <label for="username">Username: </label>
      <input type="text" id="username" name="username" placeholder="ex: joao_silva">
      <label for="name">Name: </label>
      <input type="text" id="name" name="name" placeholder="ex: João Silva">
      <label for="email">Email: </label>
      <input type="text" id="email" name="email" placeholder="ex: joao@dominio.com">
    </form>
    <button onclick="add_user()">adicionar user</button>
  `
}
```

Figura 14 - Estrutura HTML do placeholder no form do post

```
function create_post_forms(){
  const formsContent = document.querySelector('.forms-content');
  formsContent.innerHTML = '';
  formsContent.innerHTML+=`
    <form>
      <label for="title">Titulo: </label>
      <input type="text" id="title" name="title" placeholder="ex: Meu primeiro post">
      <label for="body">Texto: </label>
      <input type="text" id="body" name="body" placeholder="ex: Conteúdo do post...">
    </form>
    <button onclick="add_post()">enviar post</button>
  `
}
```

5.5 CAPITALIZAÇÃO DOS TEXTOS DOS CARDS

Foi implementada ainda a função `capitalize()`, responsável por padronizar a apresentação dos textos recebidos da API. Essa função converte a primeira letra de cada texto para maiúscula antes da exibição dos cartões, sendo aplicada tanto aos títulos quanto ao conteúdo das publicações. Como resultado, as informações são apresentadas de maneira mais organizada e visualmente consistente.

Figura 15 - Textos da tela formalizados

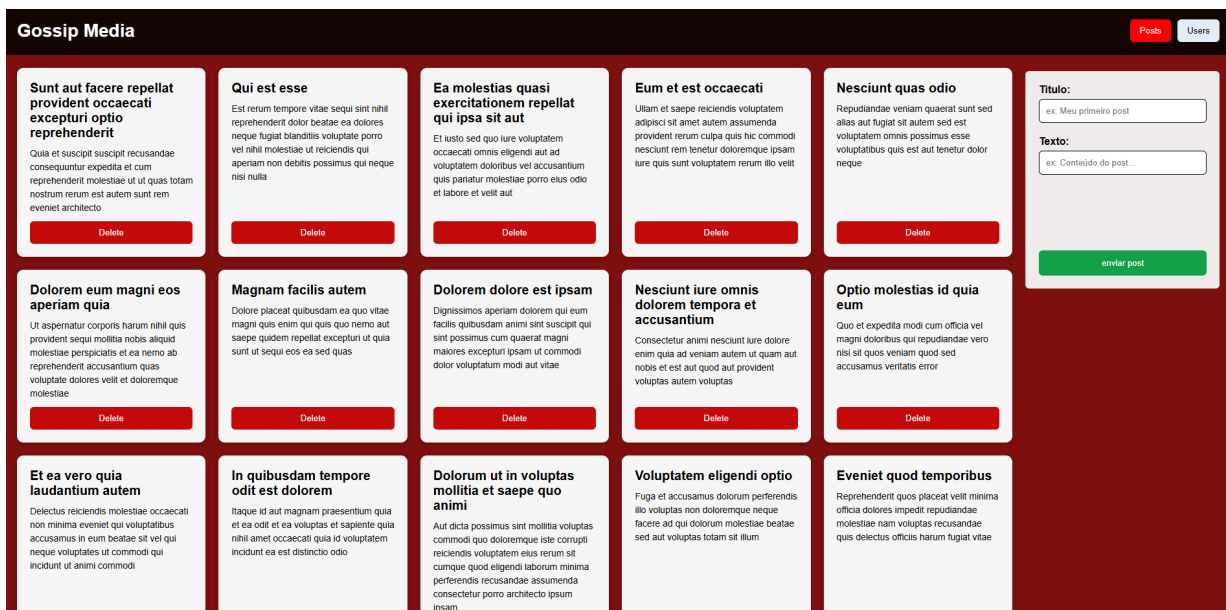


Figura 16 – Código javascript da função capitalize

```
function capitalize(str) {  
  if (!str) return '';  
  return str.charAt(0).toUpperCase() + str.slice(1);  
}
```

Figura 17 - Código javascript exemplo de uso da função capitalize

```
function create_post_cards(){  
  const cardContent = document.querySelector('.card-content');  
  cardContent.innerHTML = '';  
  dados_post.forEach(post => {  
    const card = document.createElement('div');  
    card.classList.add('card');  
    card.innerHTML = `  
      <h2>${capitalize(post.title)}</h2>  
      <p>${capitalize(post.body)}</p>  
      <button onclick="deletes(${post.id})">Delete</button>  
    `;  
    cardContent.appendChild(card);  
  });  
}
```

5.6 DESTAQUE AO BOTÃO DE NAVEGAÇÃO ATIVO

Por fim, foi adicionada uma indicação visual da seção atualmente selecionada na interface. Para isso, foram atribuídos identificadores aos botões de usuários e publicações, permitindo seu controle pelo JavaScript. Também foi criada a classe `active`, responsável por destacar o botão ativo por meio da alteração da cor de fundo para vermelho e da cor do texto para branco. A função `set_active_button()` remove inicialmente a classe de ambos os botões e, em seguida, aplica a classe apenas ao botão correspondente à seção selecionada. Essa função é executada durante o carregamento inicial da página e sempre que ocorre a alternância entre as telas, proporcionando uma navegação mais clara ao usuário.

Figura 18 - Exemplo dos botões ativos na tela



Figura 19 - Estrutura HTML com os ids

```
<ul>
  <li><button id="btn-posts" onclick="reset_window('posts')">Posts</button></li>
  <li><button id="btn-users" onclick="reset_window('users')">Users</button></li>
</ul>
```

Figura 20 - Código CSS da classe `active`

```
nav button.active {
  background-color: #ff0000;
  color: white;
}
```

Figura 21 - Código javascript da função de mudar o estado do botão

```
function set_active_button(type) {
  document.getElementById('btn-posts').classList.remove('active');
  document.getElementById('btn-users').classList.remove('active');
  if (type === 'posts') {
    document.getElementById('btn-posts').classList.add('active');
  } else {
    document.getElementById('btn-users').classList.add('active');
  }
}
```

6 QUAIS BENEFÍCIOS TROUXERAM PARA O USUÁRIO

6.1 SPINNER DE CARREGAMENTO

Elimina a incerteza durante a espera. O usuário sabe que o sistema está processando algo, em vez de achar que a aplicação travou ou não respondeu ao clique.

6.2 MODAL DE CONFIRMAÇÃO PARA EXCLUSÃO

Previne exclusões acidentais, dando ao usuário controle total sobre ações destrutivas e evitando a perda irreversível de dados causada por um clique errado.

6.3 VALIDAÇÃO DE FORMATO DE E-MAIL

Impede que dados inválidos sejam cadastrados no sistema, com mensagens claras que orientam o usuário a corrigir exatamente o campo com erro.

6.4 PLACEHOLDERS NOS CAMPOS DE FORMULÁRIO

Reduz dúvidas sobre o formato esperado em cada campo, acelerando e facilitando o preenchimento correto do formulário logo na primeira tentativa.

6.5 CAPITALIZAÇÃO DOS TÍTULOS DOS CARDS

Deixa a interface mais polida e profissional, melhorando a legibilidade e a apresentação visual dos cards de posts.

6.6 DESTAQUES DO BOTÃO DE NAVEGAÇÃO ATIVO

Melhora a orientação do usuário dentro da aplicação, deixando claro e visível em qual seção ("Posts" ou "Users") ele se encontra no momento.

CONCLUSÃO

A aplicação das Heurísticas de Nielsen permitiu identificar diferentes problemas de usabilidade presentes na interface da aplicação, evidenciando aspectos que poderiam comprometer a experiência do usuário durante sua utilização. A análise realizada demonstrou que pequenas falhas de interação, como a ausência de feedback visual, a inexistência de validações e a falta de indicações claras de navegação, podem impactar significativamente a eficiência e a intuitividade do sistema.

As melhorias implementadas atenderam aos problemas identificados, proporcionando uma interface mais segura, organizada e de fácil utilização. A inclusão do indicador de carregamento tornou o processamento das requisições mais transparente ao usuário, enquanto o modal de confirmação reduziu o risco de exclusões acidentais. Além disso, a validação do formato de e-mail, os placeholders nos formulários, a padronização dos textos dos cards e o destaque do botão de navegação ativo contribuíram para aumentar a clareza das informações, facilitar o preenchimento de dados e melhorar a orientação durante a navegação.

Dessa forma, conclui-se que a utilização das Heurísticas de Nielsen como referência para avaliação e aprimoramento da interface possibilitou o desenvolvimento de soluções que elevaram a usabilidade da aplicação. Os resultados obtidos demonstram a importância da adoção de princípios de experiência do usuário no desenvolvimento de sistemas, contribuindo para a criação de interfaces mais intuitivas e alinhadas às necessidades dos usuários.